

FinLLM: Project Handoff (context for a new Claude chat)

Paste or attach this file at the start of a new chat. It describes what exists, why it was built this way, the current results, known problems, and how I like to work.

Last updated: 27 Sep 2026.

1. Who I am and how I work

- I am Prashasti, an MSDS student building this project with my co-founder Kartik. I want to become an AI engineer / ML engineer / data scientist, and this project is my main portfolio piece.
- I write the code by hand in the terminal. I do not use Claude Code for this project.
- Please guide me step by step. Give one step at a time with a way to verify it worked, and explain the “why” briefly.
- No emojis, em dashes, arrows or other AI-looking characters in code, docs or output. Use plain text (PASS / FAIL / WARN).
- Be honest about results. Push back on weak claims, and point out when a metric is misleading.

Environment

- MacBook (Apple Silicon), zsh. Project folder: `~/FinLLM`. Python 3.11.1 in a venv at `~/FinLLM/.venv`.
- Start of every session:

```
cd ~/FinLLM
source .venv/bin/activate
python check_setup.py
```

Ollama must be running (llama icon in the menu bar).

- Repo: <https://github.com/Prashasti9/FinLLM> (private). Commit and push after every completed step.

Practical lessons about my setup

- Long terminal pastes get cut off (the prompt shows `heredoc>`). Give files as `cat > file << 'EOF'` blocks split into pieces of about 60 lines or fewer, with later pieces using `cat >>`. Press Ctrl+C if a paste gets stuck; nothing is written.
- For edits to existing files, a small Python patch script that uses `assert s.count(old) == 1` before replacing works well. Avoid long nano editing.
- macOS `sed` needs `sed -i ''`. Use `LC_ALL=C sed` for non-ASCII characters.
- Terminal cannot read `~/Downloads` (macOS privacy). Move files with Finder.
- Running the local LLM heats the laptop and drains the battery. Keep it plugged in, and avoid running two LLM jobs at once.

2. What the project is

FinLLM is a local AI assistant for anti-money-laundering (AML) compliance work. It:

1. answers questions from regulatory guidance with page citations, and refuses when the document does not cover the question (Module 1);
2. flags unusual bank transactions with rules plus an anomaly-detection model (Module 2);
3. writes alert narratives for a human analyst, where every number comes from code and the LLM only writes the prose (Module 2).

Design principle: **rules and ML detect, the LLM explains, a human decides**. Everything runs locally with Ollama, with no external AI API, because compliance teams do not want customer data sent to third parties.

Longer-term startup vision (from Kartik’s research): a “financial trust / risk” layer for fintechs. FinLLM is the proof of concept. Kartik did the idea research and an earlier separate prototype; I built the current code independently. Kartik is a collaborator

on the repo.

3. Stack

Component	Choice
LLM	qwen3:4b-instruct via Ollama (current). qwen3:4b is also installed but is the thinking-only variant (same ID as qwen3:4b-thinking)
Embeddings	nomic-embed-text via Ollama (768 dimensions)
Vector DB	ChromaDB 1.5.9, persistent at chroma_db/, collection fintech_policies
Keyword search	rank_bm25
PDF	pypdf 6.19.0
ML	pandas, scikit-learn (Isolation Forest)
Python client	ollama 0.6.2

requirements.txt: ollama==0.6.2, chromadb==1.5.9, pypdf==6.19.0, rank_bm25, pandas, scikit-learn.

4. Files

config.py	all settings (paths, models, CHUNK_SIZE=900, CHUNK_OVERLAP=150, TOP_K=8)
check_setup.py	verifies Ollama, models, embeddings, a Qwen reply, libraries, PDF
llm.py	ask_llm(prompt, system): /no_think in system prompt, think=False, options temperature 0, num_predict 700, num_ctx 8192; strips anything before </think> and any echoed /no_think
ingest.py	PDF -> pages -> 900-char chunks (150 overlap, cut at sentence ends) -> nomic embeddings with "search_document:" prefix -> ChromaDB metadata: source, page, chunk. Deterministic IDs. Deletes old chunks per file first.
ask.py	interactive Q&A. retrieve(): top 8 dense (with "search_query:" prefix) + up to 4 extra from an LLM query rewrite + up to 4 extra from BM25. New methods only ADD chunks, never replace. Numbered evidence [1]..[n], strict system prompt, exact refusal sentence, log to logs/qa_log.jsonl
eval.py	Module 1 eval. --quick runs 5 key questions (Q1, Q4, Q10, Q18, Q21). Prints seconds per question. Supports "must_not" wrong-answer phrases.
eval_questions.json	24 questions: 20 answerable (expected_pages, keywords), 4 must be refused. Q10 has must_not ["can count", "yes,"]; Q20 has must_not ["yes, there is"]
find_page.py	prints which PDF page contains a phrase (for verifying expected pages)
generate_transactions.py	synthetic data. Optional seed argument (default 42). 50 customers, 90 days from 2026-06-01, ~5,300 transactions. Planted patterns on 12 customers: structuring (3), large_new_foreign (3), velocity_burst (2), dormant_spike (2), spending_spree (2; no rule covers it). "pattern" column is the answer key.
detect.py	drops the answer key, builds per-customer features, applies rules and Isolation Forest, writes data/transactions/flags.csv
eval_detect.py	grades flags against the answer key (precision, recall, per pattern, customers caught via an actual suspicious transaction, false alarms, which layer caught each pattern)
multi_seed.py	runs generate + detect + scoring on seeds 1-5, prints mean/std/range, then restores the seed-42 data and flags
why.py	why_bullets(): computes the "WHY IT IS UNUSUAL" section exactly from reason codes
explain.py	per-customer alert: Python prints the flagged-transaction table and the WHY section; the LLM writes only SUMMARY, OPEN QUESTIONS and "Decision: requires analyst review". Checks: invented transaction IDs, amounts not in the facts, accusatory words, missing sections (one retry). Modes: no argument = top 3 by risk; a customer ID; --all prints a scorecard.
README.md	public project description with results and lessons
NOTES.md	running log of design choices, results and known issues
data/policies/	FFIEC BSA/AML Manual, Suspicious Activity Reporting section (17 pages, 82 chunks)
data/transactions/	transactions.csv, flags.csv, explanations_*.md (generated)

Detector details (detect.py)

- Features per transaction, using only the customer's past: `amount_ratio` (amount / prior median), `days_since_last`, `new_recipient` (first transfer to that payee), `foreign`, `hour`, `tx_last_hour`, `near_threshold` (cash deposit \$9,000-\$9,999) and `near_threshold_7d`, `big_outgoing` (payment/transfer at 3x usual or more) and `big_out_7d`.
- Rules (reason codes). Strong: REPEATED_NEAR_CTR_THRESHOLD, LARGE_VS_HISTORY (10x or more), NEW_RECIPIENT_LARGE, HIGH_VELOCITY (5+ in an hour), DORMANT_REACTIVATION (30+ days). Weak: NEAR_CTR_THRESHOLD, NEW_RECIPIENT, FOREIGN_DESTINATION, NIGHT_ACTIVITY (before 6am). Flag if 1+ strong or 2+ weak.
- Isolation Forest: 200 trees, contamination 0.01, random_state 42, on continuous features only: log amount, log ratio, tx_last_hour, days_since_last, night, big_out_7d. Removing the yes/no features `new_recipient` and `foreign` from the ML raised precision from 81% to 91%.
- `risk_score` = 3 x strong + weak + 2 x ML flag.

5. Results so far

Module 1 (24 questions, answers read by hand)

Model	Retrieval	Correct answers	Confident wrong answers	Refusals
qwen3:4b (thinking)	20/20	19/20	0	4/4
qwen3:4b-instruct (current)	20/20	17/20	2 (Q10, Q20)	4/4

- Q4 (“Can a bank tell the customer a SAR was filed?”) refuses with both models. The confidentiality chunk on page 15 is not retrieved. This is a safe failure.
- Q10: the instruct model says an automated alert “can count” as initial detection; page 12 says it should not.
- Q20: the instruct model says “Yes, there is a minimum amount” for insider abuse; the rule is “any amount”.
- The full eval took about 30 minutes on a hot laptop (about 75 seconds per question). Use `--quick` for everyday checks.

Module 2 detector

Metric	Seed 42 (tuned on)	Seeds 1-5 (unseen), mean +/- std
Precision	91%	92% +/- 5% (85-98%)
Recall	88%	87% +/- 8% (75-95%)
Suspicious customers caught	12/12	59/60
Innocent customers flagged	4 (C003, C011, C020, C028)	3.4 on average
Spending spree (ML only)	10/17	53% +/- 25% (15-81%)

Module 2 narratives (explain.py -all, seed 42, qwen3:4b-instruct)

- 16/16 complete, 15/16 passed automated checks (C001 mentioned T04400, which is not one of its flagged transactions), 0 retries, about 6 seconds per customer.
- Hand review of 5 (C023 real; C003, C028, C020, C011 false alarms): WHY section exact 5/5; AI summary and questions accurate 3/5; neutral 5/5. Slips: C020 “five-day period” (it was 9 days) and “frequency metrics”; C011 invented “a time when no prior transactions occurred”.

6. Key lessons (the story of the project)

1. Query rewriting first made retrieval worse (7/8 to 6/8) because distances from two different searches were sorted together. Fix: new methods only add chunks.
2. Dense search missed “initial detection” (page 12). BM25 keyword search fixed Q10 with the thinking model.
3. A stricter “never infer” prompt did not stop a 4B model from inferring. Better retrieval did.
4. Keyword scoring passed two confidently wrong answers. Added `must_not` phrases and hand review.
5. The default `qwen3:4b` tag is the thinking-only variant. That caused leaked reasoning, slowness, heat and format failures

that looked like prompt problems.

6. Ollama's default context window was 4,096 tokens, smaller than some prompts, which silently truncates the system prompt. Now 8,192.
7. The LLM copied transaction lines and ran out of output tokens. Python now prints the table and computes the WHY section.
8. Isolation Forest caught 0/17 of a rule-free pattern until a 7-day behavioral feature was added (5/17), then 10/17 after removing rare yes/no features.
9. Unsupervised ML detects "unusual", not "criminal". Narratives must describe numbers without implying wrongdoing.
10. Test on unseen data and report mean +/- spread, not one number.

Working principles we follow: set the pass criterion before running a test; change one thing at a time; re-run the eval after every change; read key answers by hand.

7. Known issues

- Instruct model: 2 confident wrong answers in Module 1. Thinking model: accurate but slow.
- Q4 retrieval gap (page 15 confidentiality chunk not retrieved).
- AI-drafted narrative summaries still contain about one small factual slip in 2 of 5 cases.
- Latency on a laptop: Module 1 full eval about 30 minutes; each question makes two LLM calls (rewrite + answer) with up to 16 chunks.
- Only one regulatory document and synthetic transactions. No supervised model yet.
- `llm.py` uses one model for everything.

8. Next steps (in suggested order)

1. **Model split:** thinking model (`qwen3:4b`) for regulation Q&A in `ask.py/eval.py`, with a higher `num_predict` so it can finish thinking and the reasoning stripped; instruct model for `explain.py` and query rewriting. Re-run the full eval and compare.
2. **Latency work, measured with `eval.py --quick`:** rewrite the query only when the best dense distance is weak; send fewer chunks (for example `5 + 2 + 2`); keep the model loaded. Track speed and accuracy together.
3. **Module 3:** link each flag to the relevant regulation page (for example, structuring flags retrieve SAR thresholds and filing deadlines from Module 1, with citations).
4. **Supervised model** on a public labeled dataset (PaySim or IBM AML) with a time-based split, PR-AUC and precision@k, compared against rules and Isolation Forest.
5. **Thin pipeline:** one command per daily batch, results and run metadata stored in SQLite; later a small Streamlit review UI.
6. **Housekeeping:** finish the README Authors section; rewrite "What I learned" in my own words. (Kartik is already a GitHub collaborator.)
7. **Validation:** show the demo to 3-5 people who work AML alerts.